

Project Workflow — Quality-First GNN Restart

Google — Fast or Slow? Predict AI Model Runtime · Solo redesign

Google — Fast or Slow? Predict AI Model Runtime · Generated 2026-06-18

- Phase 0 — Foundation ($\approx$ 2–4 days)
- Phase 1 — Data pipeline & EDA ($\approx$ 1 week)
- Phase 2 — End-to-end ranking baseline on tile:xla ($\approx$ 1 week)
- Phase 3 — Layout collections & scale ($\approx$ 1–1.5 weeks)
- Phase 4 — Solution-quality push ($\approx$ 2 weeks)
- Phase 5 — Tuning, ensembling, validation hardening ($\approx$ 1–1.5 weeks)
- Phase 6 — Finalise & document ($\approx$ 1 week)

1. Why this redesign

The original team plan (5 people, March–April) framed the competition as a **tabular regression** problem to be solved with Random Forest / XGBoost / LightGBM on a handful of hand-engineered features. That framing does not match the competition. *Google — Fast or Slow? Predict AI Model Runtime* is a **graph learning-to-rank** problem: every example is a neural-network computation graph, every graph carries many candidate compiler configurations, and we must **order those configurations from fastest to slowest**. On the public leaderboard, every competitive solution is a **Graph Neural Network (GNN)**; tree models on aggregated features can only ever serve as a weak sanity baseline.

The team has since disbanded and this is now a **solo effort**. The working folder also accumulated two half-finished, mutually inconsistent toolkits and a partial data download. This document defines a clean, **quality-first** restart whose explicit objective is a **strong final ranking**, not adherence to a fixed calendar. Phases are gated by *quality criteria*, and the timeline ($\approx$ 8–9 weeks of part-time work) flexes to meet them.

Note on competition status. The contest closed in November 2023. Submissions still score on Kaggle as *late submissions* — perfect for a learning project — but no prizes or official rank are awarded. Our

goal is the best achievable **late-submission score** and a defensible, well-documented solution.

2. Objective and success criteria

Primary objective. Maximise the competition ranking metric across all five collections, with emphasis on the **layout** collections (they dominate the overall score).

The metric is a normalised ranking quality score. For the **tile** collection it is based on the *slowdown* of the configuration we predict best relative to the true best; for the **layout** collections it is a **Kendall-tau-style** ordering score. In both cases, **only the predicted ordering matters**, never the absolute runtime — this is why we train ranking losses, not regression losses.

Definition of done for the whole project:

#	Criterion
1	A single reproducible command produces a valid 5-collection <code>submission.csv</code> .
2	Per-collection OPA (ordered-pair accuracy) on a held-out validation split is tracked and beats a trivial baseline by a wide margin.
3	Cross-validation scores correlate with the public leaderboard (no silent overfitting).
4	The final model is an ensemble with documented architecture/seed diversity.
5	A short report + figures explain the data, method, and results.

3. Guiding principles

1. **End-to-end first, optimise later.** Get a *valid* (even weak) submission through the entire pipeline before improving any single stage. A pipeline that scores 0.2 beats a brilliant model that never produces a submission.
 2. **One framework.** Commit to a single stack in Phase 0 (leaning **PyTorch + PyTorch Geometric**) and do not split effort across TensorFlow and PyTorch.
 3. **The metric is law.** Wire **OPA / Kendall-tau** evaluation in from day one and judge every change by it. Never trust training loss alone.
 4. **Trustworthy validation.** Build a cross-validation scheme whose score *moves with* the leaderboard. Quality work is impossible without a reliable offline signal.
 5. **Reproducibility by construction.** Every run is driven by a config file, seeds are fixed, and artifacts (checkpoints, submissions, metrics) are written to `artifacts/`.
 6. **Smoke-test on tiny data.** Every new component must first run on a 5-file subset in seconds before touching the full dataset.
 7. **Protect the labels.** Never clip, normalise, or "clean" `config_runtime` in a way that changes the *ordering* of configurations — that silently destroys the target. (The old `data_cleaning.py` clipped runtimes; it is archived and must not be reused as-is.)
-

4. Repository structure

```
kaggle---prj/
  README.md  requirements.txt  .gitignore
  WORKFLOW.pdf  STUDY_GUIDE.pdf      # the two deliverables (this file + the guide)
  data/      # downloaded NPZ – never edited by code
  docs/
    src/      # markdown sources + build_pdf.py for the PDFs
    USER_GUIDELINE.md  DATA_STRUCTURE.md # dataset references
  notebooks/      # the official TF-GNN starter + future EDA notebooks
  src/
    data/      # NPZ readers, graph construction, batching, caching, CV splits
    features/   # feature engineering / normalisation statistics
    models/     # GNN architectures (GCN, GraphSAGE, GAT, GPS), heads
    training/   # losses (ListMLE, pairwise), metrics (OPA), train loop
    inference/  # prediction + submission-file assembly
    utils/     # seeding, config, logging
  configs/     # one YAML per experiment (model, data, optimiser, seed)
  scripts/     # CLI entrypoints: make_inventory.py, train.py, predict.py, make_submission.py
  artifacts/
    figures/   checkpoints/  submissions/
  tests/      # pytest: loader shapes, metric correctness, submission format
  archive/
    root-scripts/ # original team scripts (reference only)
    ref-toolkit/  # the second toolkit design (reference only)
```

Experiment flow. A run is one command:

```
config (YAML) → scripts/train.py → trains a model, logs OPA per epoch
              → best checkpoint in artifacts/checkpoints/
              → scripts/predict.py → per-collection ranking
              → scripts/make_submission.py → artifacts/submissions/<name>.csv
              → leaderboard log (CV score, LB score, notes)
```

5. The phased plan

Each phase has a **goal**, **key tasks**, and a **quality gate** — a concrete, checkable exit criterion. Do not advance until the gate is met. Time estimates assume part-time solo work and are deliberately soft.

Phase 0 — Foundation ($\approx$ 2–4 days)

Goal. A clean, reproducible workspace and a final framework decision.

- Confirm the clean folder layout (this document) and create `requirements.txt`, `.gitignore`.
- **Framework decision spike:** stand up a *minimal* "hello graph" in both candidates if unsure — load one NPZ, build a graph object, run one message-passing layer — and pick the winner. Default and recommendation: **PyTorch + PyTorch Geometric** (largest TPUGraphs community, easiest debugging). Record the decision in `README.md`.
- **Data integrity audit:** confirm all five collections are present and intact. Current state: **all five are downloaded** — `tile/xla`, `layout/nlp/{random,default}`, and `layout/xla/{random,default}` ($\approx$ 160 layout/xla files, multi-MB each, no zero-byte files). Verify file counts and schema hashes (no re-download expected). Normalise the on-disk path convention to a single root and document it.
- Fix seeds utility, config loader, logging skeleton.

Quality gate P0: from a fresh checkout, `pip install -r requirements.txt` succeeds, the chosen framework imports, and a script prints the shapes of one tile and one layout NPZ. All five collections are present on disk.

Phase 1 — Data pipeline & EDA ($\approx$ 1 week)

Goal. Robust, fast, *correct* data loading for both task families.

- **Inventory** every NPZ (collection, split, `n_nodes`, `n_edges`, `n_configs`, schema hash, NaN/Inf flags, edge-index bounds) into a single parquet. Reuse the logic in `archive/ref-toolkit/inventory_generation.py` as a reference.
- **Graph construction:** convert raw arrays to the framework's graph object. Pin the conventions once and for all: `edge_index` orientation, dtypes, and **directionality** (information flows along the *transpose* of the `feed` adjacency — see the study guide).
- **Variable-size batching:** graphs differ wildly in size; implement correct mini-batching (PyG `Batch` / a custom ranking collator). Verify it works at batch size > 1 — the old `enhanced_loader.py` silently broke here.
- **Config sampling:** during training, sample a fixed number of configurations per graph (e.g. 8–32) to form ranking lists; keep all configs at inference.
- **On-disk caching:** preprocess once to a fast cache (the $\sim$ 7k tile files and the huge layout graphs are slow to re-read every epoch).

- **Normalisation statistics:** compute train-split node-feature means/stds (and a `log1p` transform for heavy-tailed size features); store to `artifacts/`. **Fit on train only.**
- **EDA figures:** distributions of `n_nodes`, `n_configs`, runtime ranges per collection.

Quality gate P1: a `DataLoader` yields correctly-batched graphs for **every** collection at batch > 1; node features are finite after normalisation; cached epoch iteration is markedly faster than cold reads; `pytest` covers loader shapes and edge-index bounds.

Phase 2 — End-to-end ranking baseline on `tile:xla` ($\approx$ 1 week)

Goal. The first complete, valid submission, on the easiest collection.

- **Cross-validation split** by graph (never leak configs of one graph across folds).
- **Metric:** implement **OPA** and a Kendall-tau check; unit-test against a tiny hand-worked example.
- **Losses:** implement **pairwise hinge** and **ListMLE**; start with pairwise for stability.
- **Model:** a small message-passing GNN — op-code **embedding** then concat config features, 2–3 graph-conv layers, global pooling, one score per configuration.
- **Trivial baselines** for sanity: random order, and rank by a single raw config feature. The GNN must beat both clearly.
- **Submission assembly:** produce the `ID,TopConfigs` format and validate it against the sample submission.

Quality gate P2: a trained GNN beats the trivial baselines on held-out OPA; a valid tile-only `submission.csv` is produced end-to-end by one command; CV is reproducible.

Phase 3 — Layout collections & scale ($\approx$ 1–1.5 weeks)

Goal. Handle the big, high-value layout graphs and produce a full submission.

- Validate `layout/xla/*` integrity (already on disk; confirmed in Phase 0).
- **Layout loader:** only a subset of nodes are configurable (`node_config_ids`); config features are **per configurable node** (`node_config_feat [c, nc, 18]`). Scatter them onto the right nodes; pad/mask non-configurable nodes.
- **Graph Segment Training / segment dropout:** layout graphs reach tens of thousands of nodes and will not fit otherwise. Implement segment sampling (keep a contiguous subset of nodes, run a full-graph forward pass under `stop_gradient`, backprop through the kept segment). This is the single most important technique for layout — see the study guide.
- Train one model per layout collection (`xla:random`, `xla:default`, `nlp:random`, `nlp:default`); these distributions differ, so per-collection models usually win.
- **Assemble the full 5-collection submission** by concatenating all per-collection rankings.

Quality gate P3: layout models train without OOM via segment training and beat trivial baselines on each collection; a **full 5-collection** submission scores on the public leaderboard; the CV→LB gap is understood and reasonable.

Phase 4 — Solution-quality push (≈ 2 weeks)

Goal. Move from "works" to "competitive". This is where most score is won.

- **Study published top solutions** (winners' write-ups, the TPUGraphs paper). Reproduce one high-value idea at a time and keep it only if CV improves.
- **Feature engineering** (auxiliary, fed into the GNN, *not* replacing it):
 - `log1p` of size/shape node features; standardise the rest.
 - Richer **op-code embeddings**; consider embedding dimensionality sweeps.
 - **Config-feature encoding** (the 18-d layout / 24-d tile config vectors): per-feature normalisation, interactions, learned encoders.
 - **Structural features**: in/out degree, topological depth approximation, subgraph id.
- **Architecture exploration**: GraphSAGE, GAT (attention), GPS / transformer-conv, deeper message passing with residual connections, and a **virtual/global node** for long-range pooling. Compare under identical CV.
- **Loss tuning**: compare ListMLE vs pairwise vs a hybrid; tune the number of sampled configurations per list.

Quality gate P4: at least one architecture or feature change yields a *statistically meaningful* CV gain over the Phase-3 model on the layout collections, logged with before/ after numbers.

Phase 5 — Tuning, ensembling, validation hardening ($\approx 1-1.5$ weeks)

Goal. Squeeze and stabilise.

- **Hyperparameter search** (learning rate, hidden dim, #layers, embedding dim, dropout, sampled-config count) — coarse random search guided by CV.
- **Ensembling**: average rankings across **seeds** and **architectures** per collection (rank averaging is robust for ranking tasks). Verify each member helps the ensemble.
- **CV ↔ LB correlation analysis**: confirm offline improvements transfer; if not, fix the validation scheme before trusting any further gains.
- **Error analysis**: which graphs/collections are worst, and why.

Quality gate P5: the ensemble beats the best single model on CV *and* LB; the CV→LB relationship is documented and monotone enough to trust.

Phase 6 — Finalise & document ($\approx$ 1 week)

Goal. A clean, reproducible, well-explained final result.

- Freeze the final config(s); regenerate the final submission from scratch to prove reproducibility.
- Write the report (problem, data, method, results, ablations) with figures from `artifacts/figures/`.
- Tidy code, ensure `pytest` is green, update `README.md` with exact repro commands.

Quality gate P6: one documented command reproduces the final submission; report and figures are complete.

6. Milestones at a glance

Phase	Outcome	Gate (must hold to advance)
P0	Clean repo, framework chosen, all data present	Fresh install runs; 5 collections on disk
P1	Correct, cached, batched loaders + EDA	Batch>1 works for every collection; finite features
P2	Tile GNN + first valid submission	GNN beats trivial baseline on OPA; reproducible CV
P3	Layout via segment training; full submission	No OOM; 5-collection LB score; sane CV→LB gap
P4	Competitive model	Meaningful CV gain on layout
P5	Tuned ensemble	Ensemble > best single on CV and LB
P6	Final, reproducible, documented	One-command repro; report done

7. Risk register

Risk	Impact	Mitigation
Layout graphs cause out-of-memory	Blocks the highest-value collections	Graph Segment Training; cap kept nodes; gradient accumulation; mixed precision
Corrupt or partial NPZ files	Silent training failures	Integrity audit (file counts, schema hash, finite-value checks) in Phase 0; re-download if needed
Label corruption from old cleaning (runtime clipping)	Silently wrong targets	Never alter <code>config_runtime</code> ordering; archived cleaner is reference-only
CV ↔ LB divergence	Optimising the wrong thing	Group-by-graph CV; correlation check in P5; conservative changes
Framework thrash (TF vs PyTorch)	Wasted effort	Decide once in P0 and commit
Variable-size batching bugs	Wrong gradients, silent score loss	Unit tests on shapes/bounds; verify at batch>1 early
Solo scope creep	Never finishing	Phase gates; end-to-end before optimisation; one idea at a time in P4

8. Tooling decisions

- **Language/stack:** Python; **PyTorch + PyTorch Geometric** (recommended) for the GNN.
 - **Metric/loss:** OPA + Kendall-tau evaluation; ListMLE and pairwise-hinge losses.
 - **Config:** YAML per experiment in `configs/` ; fixed seeds via a `utils/seed` helper.
 - **Experiment tracking:** a simple CSV/markdown leaderboard log under `artifacts/` (model, config hash, CV-OPA per collection, LB score, notes). Upgrade to W&B only if needed.
 - **Testing:** `pytest` for loaders, metrics, and submission format.
 - **Reference material:** the official starter notebook in `notebooks/` and the two archived toolkits in `archive/` — read, don't run blindly.
-

9. Immediate next step

Begin **Phase 1: the data-loading stage** — build the NPZ inventory and the correct, cached, variable-size graph loaders in `src/data/`, with `pytest` coverage. Everything downstream depends on this being correct, so it is the first code we write.